曦望二面：推理引擎与整网优化

> 技术面复盘 · 插件适配 / TP 与 PP / GEMM 分块 / Attention / KV Cache / 投机解码。问题依据转写改写，公司与轮次沿用来源文件标注。

一、这一轮主要考什么

这一轮强调推理引擎的系统视角。面试不满足于“写过一个更快的 Kernel”，还要求说明并行方式、通信、调度、缓存和模型结构如何共同影响实际服务性能。

二、源记录中可确认的提问

1. 新硬件接入 vLLM 插件体系，需要替换或扩展哪些能力？
2. 相同卡数下，TP 与 TP+PP 的性能差异应怎样分析？
3. TP、PP 和 EP 各涉及哪些通信行为，不能混用哪些通信库概念？
4. batch 增大后，GEMM 反而更快，怎样继续追问 Kernel 层根因？
5. 分块策略为什么会造成不同 shape 下的性能差异？
6. FlashAttention 的主要优化是什么？与计算量、数据搬运分别有什么关系？
7. Online Softmax 如何逐块更新，为什么能够保持数学等价？
8. 对 GQA、MLA 和其他注意力变体了解多少？哪些只是看过名词？
9. TP、PP 等组合下，KV Cache 应怎样分配？
10. 如何理解 KV Cache 的多级缓存，和硬件缓存层级有什么区别？
11. 投机解码在不同负载或 batch 下是否一定有收益？怎样判断？
12. 推理引擎优化与基础算子优化，各需要补哪些能力？

三、整理补充：三道情景题怎样展开

情景 A：更换并行方式后性能下降

先固定模型、请求长度、并发、输出长度、精度和硬件拓扑，再列出层切分、通信、流水线气泡和调度差异。准备查看哪个 timeline、怎样测每阶段耗时，以及哪一个实验可以推翻当前假设。不要直接认定“通信多所以慢”。

情景 B：小 batch 的 GEMM 比大 batch 更慢

先区分整网延迟和已定位 Kernel 的时间。若问题已被限定在 Kernel 内，应继续检查具体选择的实现、tile、padding、有效工作量、访存与资源占用，而不是反复回到引擎级开关。每个候选原因都配一项验证方法。

情景 C：缓存和投机解码如何落到服务指标

先画请求路径，注明缓存驻留位置、传输路径、命中时与未命中时分别做什么。投机解码准备记录 draft 成本、验证成本、接受情况、并发以及延迟和吞吐。避免脱离负载条件判断某方案必然更快。

四、补课与练习清单

- 阅读目标版本插件文档，列出平台、worker、通信、Attention 和算子适配项。
- 手推一行 Softmax 分成两块后的统计量合并，解释已有结果如何重标定。
- 画出注意力部分与专家部分，分别标注其参数和缓存布局。
- 做一张“算子证据 → 单层证据 → 服务指标”的验证表。

这些是整理者提出的复习任务，不是原面试答案。公开技术校准与参考入口见本目录《阅读说明与基础校准》。

五、来源与保留边界

来源：`曦望AI INFRA2面_转写.txt`。已删去自我介绍、个人经历、岗位选择和流程沟通。原转写存在模型名、库名与术语误识，本篇仅按明确上下文统一常见术语；不照搬回答中不准确的技术断言，也不补写候选人当时没有回答出来的内容。